

AIQT CLI

Milestone 7 Build Specification v0.3

Guided Workflow and Agent-Assisted Inputs

Build-planning document for implementing AIQT Milestone 7. Not a runtime document model for AIQT user projects.

Field	Value
Product	AIQT CLI
Specification	Milestone 7 Build Specification
Version	v0.3
Milestone	M7 - Guided Workflow and Agent-Assisted Inputs
Aligns With	AIQT MVP Specification v0.6 and Technical Architecture Specification v0.3
Depends On	M1 - Canonical State Foundation, tag m1-canonical-state-foundation M2 - Project Context Capture, tag m2-project-context-capture M3 - Work Graph Planning Foundation, tag m3-work-graph-planning M4 - Agent Handoff Packet Engine, tag m4-agent-handoff-packet M5 - Checkpoint Engine, tag m5-checkpoint-engine M6 - Review and Export Engine, merge commit bd833ae343f3fbd16ac5747b5605a54ee2861a42, tag m6-review-export-engine
Primary Agent Target	Claude Code, Codex, or equivalent coding agent
Implementation Status	Ready for technical review before build handoff
Document status	v0.3 review candidate. Resolves exit-code consistency, priority-ordered guidance, shared workflow precedence reuse, canProceedWithoutAgent semantics, and prompt --out overwrite behavior.

Document status: v0.3 review candidate. Resolves exit-code consistency, priority-ordered guidance, shared workflow precedence reuse, canProceedWithoutAgent semantics, and prompt --out overwrite behavior.

Contents

- 1. Source Alignment
- 2. Milestone Objective
- 3. Guided Workflow Model Decision
- 4. Commands in Scope
- 5. Out of Scope
- 6. Behavioral Position in the Workflow
- 7. User Experience Target
- 8. Guided Navigation Rules
- 9. Prompt Generation Model
- 10. Prompt Input and Output Contracts
- 11. Import Model
- 12. Local Input Workspace
- 13. Command Behavior
- 14. Data and Schema Contracts
- 15. Canonical Mutation Rules
- 16. Runlog Rules
- 17. Error Handling and Exit Codes
- 18. Repository Implementation Scope
- 19. Runtime and Package Impact
- 20. Testing Scope
- 21. Implementation Tasks
- 22. Definition of Done
- 23. Agent Execution Packet
- Appendix A - Minimum Test Matrix
- Appendix B - Example Generated Prompts
- Appendix C - Example JSON Outputs

1. Source Alignment

This document is derived from AIQT MVP Specification v0.6, AIQT Technical Architecture Specification v0.3, and the completed milestone sequence M1 through M6. It assumes M6 is complete, merged to main at `bd833ae343f3fbd16ac5747b5605a54ee2861a42`, tagged as `m6-review-export-engine`, and validated with 321/321 tests passing before and after merge. M7 may now be reviewed as a build-handoff candidate, but it still requires Claude review before implementation.

M6 review/export baseline: `aiqt review` is read-only; `aiqt export` writes only generated markdown under `.aiqt/exports/`; checkpoint packet-reference integrity is validated using `state.lastAgentPacket.id` plus packet IDs from valid `agent_packet.created` runlog events.

The governing design remains:

```
state -> graph -> packet -> checkpoint -> review/export
not:
spec -> milestone file -> task file -> prompt file -> checkpoint file
```

M7 does not replace the strict engine built by M1 through M6. It adds a guided user-experience layer that helps a human project owner and an external coding agent produce the structured inputs required by the existing engine.

1.1 v0.3 Change Summary

- Updates M7 source alignment to the completed M6 baseline: merge commit `bd833ae343f3fbd16ac5747b5605a54ee2861a42`, tag `m6-review-export-engine`, and 321/321 tests passing.
- Carries forward the M6 packet-reference review fix: checkpoint packet integrity is validated using `state.lastAgentPacket` plus `agent_packet.created` runlog packet history.
- Introduces a guided workflow layer so a user can start from an empty project folder without hand-authoring JSON files.
- Adds deterministic prompt generation for update, plan, and checkpoint input creation.
- Adds optional import delegation so agent-generated JSON can be validated and applied through existing update, plan, and checkpoint services.
- Adds `aiqt start` and `aiqt continue` as read-only workflow navigators.
- Introduces `.aiqt/inputs/` as an optional, non-canonical, generated working area for prompts and agent-produced JSON inputs; preserves state-first design and excludes Claude/OpenAI/Codex API calls, autonomous planning, source-code modification, and hosted workflow orchestration.
- Changes missing `.aiqt/` behavior for `start/continue/prompt/import` to `status = failed` and `exitCode = 3`, matching the Technical Architecture and M1-M6 precedent.
- Declares guided navigation rules priority-ordered: evaluate top-to-bottom and the first matching row wins.
- Requires M7 guidance to reuse `state.nextRecommendedCommand`, existing workflow helpers, and M6 review next-command precedence where applicable rather than creating a divergent workflow-decision engine.
- Defines `canProceedWithoutAgent` deterministically and defines prompt `--out` overwrite behavior: existing files are not overwritten by default and return exit code 2.

2. Milestone Objective

Milestone 7 implements guided workflow and agent-assisted input support. Its purpose is to make AIQT usable from an empty folder by guiding the user through the existing strict workflow and producing copy-paste prompts for Claude Code, Codex, or equivalent agents.

M7 must prove that AIQT can:

- inspect the current canonical workflow state and tell the user the next useful step;
- generate deterministic prompts for creating update, plan, and checkpoint JSON;
- optionally write generated prompts under `.aiqt/inputs/` without making them canonical state;
- validate and import agent-generated JSON through the existing engine pathways;
- avoid forcing the user to manually design the JSON schemas by hand;

- keep normal workflow state inside project.json, state.json, and runlog.jsonl;
- preserve all M1, M2, M3, M4, M5, and M6 behavior.

3. Guided Workflow Model Decision

M7 treats guidance as a deterministic local CLI function, not as AI execution. AIQT can generate prompts that a user pastes into Claude Code or Codex. AIQT does not call the agent, does not receive live agent responses, and does not alter project source code.

Guidance-source rule: M7 guidance must reuse existing workflow state helpers, state.nextRecommendedCommand, and the M6 review next-command precedence where applicable. M7 may format guidance differently, but it must not create an independent second workflow-decision engine that can drift from aiqt review, aiqt next, aiqt status, update, plan, or checkpoint behavior.

Decision	Reason
Guidance is read-only by default.	The user should be able to run aiqt start or aiqt continue safely at any time to understand the workflow position.
Prompts are generated from canonical state.	The agent receives bounded instructions based on the same source of truth used by the engine.
Input files are optional working artifacts.	They improve UX but do not become canonical state or a new markdown-first workflow.
Import delegates to existing services.	M7 must not reimplement update, plan, or checkpoint logic. It should route validated inputs through existing command/service pathways.

4. Commands in Scope

```
aiqt start
aiqt start --json
aiqt continue
aiqt continue --json
aiqt prompt update
aiqt prompt update --json
aiqt prompt update --out .aiqt/inputs/update.prompt.md
aiqt prompt plan
aiqt prompt plan --json
aiqt prompt plan --out .aiqt/inputs/plan.prompt.md
aiqt prompt checkpoint
aiqt prompt checkpoint --json
aiqt prompt checkpoint --out .aiqt/inputs/checkpoint.prompt.md
aiqt import update --from-file <path>
aiqt import update --from-file <path> --json
aiqt import plan --from-file <path>
aiqt import plan --from-file <path> --json
aiqt import checkpoint --from-file <path>
aiqt import checkpoint --from-file <path> --json
```

The prompt commands generate instructions for an external agent. The import commands are convenience wrappers over existing update, plan, and checkpoint behavior. They must not create a parallel mutation path.

5. Out of Scope

- Claude, OpenAI, Codex, GitHub Copilot, or GitHub API integration;
- automatic agent execution;

- automatic source-code modification;
- autonomous AI planning inside AIQT;
- web UI, hosted backend, or cloud synchronization;
- replanning or changing work graph structure beyond existing M3/M6 rules;
- new canonical state files such as session.json or inputs.json;
- persistent markdown source-of-truth files;
- AIQT.md, AGENTS.md, CLAUDE.md, docs/, .milestones/, or .tasks/ generation;
- changing the update, plan, next, checkpoint, review, export, or status contracts except where explicitly routed by import convenience commands;
- schema migrations or AIQT_SCHEMA_VERSION bump unless implementation proves unavoidable.

6. Behavioral Position in the Workflow

M1: init/status/next -> M2: update -> M3: plan -> M4: next packet -> M5: checkpoint -> M6: review/export -> M7: guided workflow UX

After M6, the feature-complete engine exists on main at bd833ae343f3fbd16ac5747b5605a54ee2861a42 and tag m6-review-export-engine. The human still has to coordinate structured input creation with an external agent. M7 fills that usability gap. It does not introduce a new core workflow stage; it provides a guided layer around the existing workflow.

The preferred human experience becomes:

```
mkdir my-project
cd my-project
aiqt init
aiqt start
```

From there, AIQT should tell the user exactly which prompt to generate, which agent to ask, where to save the returned JSON, and which existing AIQT command will apply it.

7. User Experience Target

Current strict engine flow	M7 guided flow
User manually prepares update.json.	aiqt prompt update generates a prompt that asks the agent to return valid update JSON.
User manually prepares plan.json.	aiqt prompt plan generates a prompt with current project context and the exact required plan JSON contract.
User manually prepares checkpoint.json.	aiqt prompt checkpoint generates a prompt using current work unit and last packet metadata.
User remembers next command.	aiqt start or aiqt continue shows the next guided action and follow-up command.
User runs update/plan/checkpoint --from-file directly.	User may run aiqt import <type> --from-file <path>, which delegates to the existing engine.

8. Guided Navigation Rules

aiqt start and aiqt continue use the same deterministic navigation engine. The two commands are intentionally equivalent in M7; start is friendlier for a new project, continue is friendlier for a resumed project.

Guided navigation rules are evaluated top-to-bottom. The first matching row wins. Later rows must not override an earlier match. This is intentional for overlapping states such as currentWorkUnitId being non-null while another work unit also needs review; in-progress work must continue to recommend aiqt checkpoint first.

Workflow condition	Guided recommendation	Exit code
.aiqt/ missing	Run aiqt init first.	3
Initialized but planningContextReady is false	Run aiqt update or aiqt prompt update.	0
Planning context ready and workGraph is empty	Run aiqt prompt plan, then aiqt import plan --from-file .aiqt/inputs/plan.json.	0
Work graph has ready work and currentWorkUnitId is null	Run aiqt next and paste the packet into the coding agent.	0
currentWorkUnitId references an in_progress work unit	Run aiqt prompt checkpoint after the agent reports results, then aiqt import checkpoint --from-file .aiqt/inputs/checkpoint.json.	0
One or more work units need review	Run aiqt review and resolve review findings manually or through a future workflow.	0
All work done	Run aiqt review, then aiqt export all.	0
Canonical files invalid or unsupported	Run aiqt status; repair or upgrade AIQT.	3

Normal workflow positions are not failures. start/continue should be safe to run in CI and during human inspection.

M7 may guide the user based on aiqt review output, but it must not mutate state to resolve review findings. Review/export behavior remains owned by M6.

9. Prompt Generation Model

Prompt generation creates copy-paste instructions for an external AI agent. The prompt must be specific enough that the agent can return one JSON object matching the appropriate existing input schema. Prompt output is not a canonical file.

Prompt type	Purpose	Required state
update	Create or refine project context JSON for aiqt update/import update.	Initialized project. No graph required.
plan	Create structured work graph JSON for aiqt plan/import plan.	planningContextReady = true and workGraph empty.
checkpoint	Create checkpoint JSON from the coding agent result.	currentWorkUnitId points to an in_progress work unit and lastAgentPacket exists.

9.1 Prompt content rules

- Prompts must state that the agent should return JSON only, with no markdown wrapper, unless the user explicitly asks for explanation.
- Prompts must include the exact expected top-level fields for the relevant input type.
- Prompts must include compact current project context, not full runlog history.
- Prompts must include the follow-up AIQT command to run after saving the JSON.
- Prompts must avoid instructing the agent to modify source code during update or plan prompt generation.
- Checkpoint prompts may include a placeholder for the user to paste the agent implementation report.
- Prompts must not include secrets, environment variables, or private local file contents beyond canonical AIQT state.

10. Prompt Input and Output Contracts

Flag	Behavior
--json	Return CommandResult JSON with data.prompt rather than raw prompt text.
--out <path>	Write the prompt to the supplied path if and only if the path is under .aiqt/inputs/ and the target file does not already exist. Create .aiqt/inputs/ lazily. Existing files are not

	overwritten by default.
No --out	Print the prompt to stdout. No file mutation.
Unsupported prompt kind	Exit code 3. No mutation.
Prompt requested in invalid workflow position	Exit code 2. No mutation. Return the correct next guidance.

10.1 PromptResultData

```
export interface PromptResultData {
  promptKind: "update" | "plan" | "checkpoint";
  prompt: string;
  suggestedOutputPath: string;
  followUpCommand: string;
  stateSummary: {
    planningContextReady: boolean;
    projectStatus: string | null;
    currentMilestoneId: string | null;
    currentWorkUnitId: string | null;
    nextRecommendedCommand: string | null;
  };
  wroteFile: boolean;
  outputPath: string | null;
}
```

10.2 Prompt --out overwrite behavior

If --out is supplied and the target path does not exist, M7 creates the file atomically. If the target path already exists, M7 must not overwrite it by default; return status = blocked, exitCode = 2, no mutation, and nextRecommendedCommand set to the same aiqt prompt command with a different --out path. Optional --force overwrite is out of scope for M7.

11. Import Model

aiqt import is a convenience router. It must not define new update, plan, or checkpoint semantics. It reads the provided file, identifies the declared import type from the command, and delegates to the same service logic used by the existing commands.

Command	Delegates to	Mutation rules
aiqt import update --from-file <path>	aiqt update --from-file <path>	Same as M2 update.
aiqt import plan --from-file <path>	aiqt plan --from-file <path>	Same as M3 plan.
aiqt import checkpoint --from-file <path>	aiqt checkpoint --from-file <path>	Same as M5 checkpoint.

Import must not auto-detect types in M7. The user must specify update, plan, or checkpoint. This avoids ambiguous validation when two schemas happen to partially parse.

12. Local Input Workspace

.aiqt/inputs/ is an optional working area for prompts and agent-produced input JSON. It is not part of the canonical source of truth. AIQT init must not create it. M7 prompt commands may create it only when --out is used.

Path	Purpose	Canonical?
.aiqt/inputs/update.prompt.md	Generated prompt asking an agent to produce update JSON.	No
.aiqt/inputs/update.json	Suggested location for agent-produced update input.	No

.aiqt/inputs/plan.prompt.md	Generated prompt asking an agent to produce plan JSON.	No
.aiqt/inputs/plan.json	Suggested location for agent-produced plan input.	No
.aiqt/inputs/checkpoint.prompt.md	Generated prompt asking an agent to produce checkpoint JSON.	No
.aiqt/inputs/checkpoint.json	Suggested location for agent-produced checkpoint input.	No

The inputs folder is intentionally disposable. Deleting it must not break normal AIQT operation.

13. Command Behavior

13.1 aiqt start / aiqt continue

Locate .aiqt/. If missing, return status = failed, exitCode = 3, nextRecommendedCommand = aiqt init, and no mutation.

1. Read project.json and state.json.
2. Validate schemas and versions.
3. Inspect runlog health using existing non-blocking behavior where possible.
4. Evaluate planningContextReady using the M2 helper.
5. Inspect workGraph, currentWorkUnitId, work-unit statuses, checkpoints, and lastAgentPacket.
6. Return deterministic guidance using Section 8.
7. Do not mutate project.json, state.json, runlog.jsonl, exports, or inputs.

13.2 aiqt prompt update

8. Locate and validate .aiqt/.
9. Read project.json and state.json.
10. Render an update prompt from current project context.
11. If --out is supplied, validate that the path is under .aiqt/inputs/ and write the prompt atomically.
12. Return prompt text in human mode or CommandResult JSON in --json mode.
13. Do not mutate canonical state or append runlog events.

13.3 aiqt prompt plan

14. Locate and validate .aiqt/.
15. Verify planningContextReady is true.
16. Verify state.workGraph is empty; if not empty, block and recommend aiqt next/review according to current state.
17. Render a plan prompt with project objective, target users, constraints, requirements, decisions, risks, assumptions, open questions, quality expectations, and the M3 plan input contract.
18. If --out is supplied, validate that the path is under .aiqt/inputs/ and write the prompt atomically.
19. Do not mutate canonical state or append runlog events.

13.4 aiqt prompt checkpoint

20. Locate and validate .aiqt/.
21. Verify currentWorkUnitId points to an in_progress work unit.
22. Verify lastAgentPacket exists and references the current work unit.
23. Render a checkpoint prompt including the current work unit, acceptance criteria, validation commands, packet metadata, and a placeholder for the agent implementation report.
24. If --out is supplied, validate that the path is under .aiqt/inputs/ and write the prompt atomically.
25. Do not mutate canonical state or append runlog events.

13.5 aiqt import

26. Validate import type is update, plan, or checkpoint.
27. Validate --from-file is supplied; if missing, return exit code 10.

28. Delegate to the corresponding existing service/command behavior.
29. Return the delegated CommandResult semantics with M7 import context in data.import if JSON mode is requested.
30. Do not add any mutation beyond the delegated command mutation.

14. Data and Schema Contracts

14.1 WorkflowAction extension

M7 introduces additional public commands. The internal WorkflowAction type may be extended with start, continue, prompt, and import. This is not a canonical file schema change because CommandResult is runtime output, not stored project state.

```
export type WorkflowAction =  
  | "init" | "update" | "plan" | "next" | "checkpoint"  
  | "review" | "export" | "status"  
  | "start" | "continue" | "prompt" | "import";
```

14.2 GuidanceResultData

```
export interface GuidanceResultData {  
  stage:  
    | "not_initialized"  
    | "needs_context"  
    | "needs_plan"  
    | "ready_for_handoff"  
    | "awaiting_checkpoint"  
    | "needs_review"  
    | "ready_for_export"  
    | "invalid_state";  
  guidance: string;  
  recommendedCommand: string | null;  
  alternativeCommands: string[];  
  promptCommand: string | null;  
  expectedInputPath: string | null;  
  followUpCommand: string | null;  
  canProceedWithoutAgent: boolean;  
}
```

14.2.1 canProceedWithoutAgent rule

canProceedWithoutAgent = true when the next action can be completed directly by the user without asking Claude/Codex to generate structured input. Examples: aigt update interactive mode is available; aigt next can produce a packet; aigt review can inspect state; aigt export can generate reports.

canProceedWithoutAgent = false when the next action normally requires agent-assisted structured input. Examples: aigt plan needs a plan JSON file; aigt checkpoint needs a checkpoint JSON file unless a checkpoint input file already exists at the expected path.

14.3 ImportResultData

```
export interface ImportResultData {  
  importType: "update" | "plan" | "checkpoint";  
  sourcePath: string;  
  delegatedAction: "update" | "plan" | "checkpoint";  
  followUpCommand: string | null;  
}
```

15. Canonical Mutation Rules

Command	Allowed mutation
aiqt start / aiqt continue	None. Read-only.
aiqt prompt <kind>	None unless --out is supplied; then only writes the prompt file under .aiqt/inputs/. No canonical state mutation.
aiqt import update	Same as aiqt update --from-file.
aiqt import plan	Same as aiqt plan --from-file.
aiqt import checkpoint	Same as aiqt checkpoint --from-file.

- M7 must not mutate project.json, state.json, or runlog.jsonl during start, continue, or prompt generation.
- M7 must not create AIQT.md, AGENTS.md, CLAUDE.md, docs/, .milestones/, or .tasks/.
- M7 may create .aiqt/inputs/ lazily only when writing prompt output by explicit --out.
- M7 must not write agent-generated JSON itself unless a user-provided file is explicitly imported through aiqt import.
- AIQT_SCHEMA_VERSION remains 0.5.0 unless implementation proves a canonical file schema change is unavoidable.

16. Runlog Rules

- aiqt start and aiqt continue append no runlog events.
- aiqt prompt appends no runlog events, even when --out writes a prompt file.
- aiqt import does not create a new import-specific runlog event in M7.
- aiqt import uses the delegated command runlog behavior: project.updated for update, work_graph.generated for plan, checkpoint.created and status events for checkpoint.
- A failed or blocked import must not append runlog events beyond the delegated command behavior.

17. Error Handling and Exit Codes

Missing .aiqt/ is not a valid initialized workflow state. All M7 commands that require an initialized project must return status = failed, exitCode = 3, nextRecommendedCommand = aiqt init, and no mutation when .aiqt/ or canonical project/state files are missing. Exit code 2 is reserved for valid initialized projects where the requested guided action is blocked by the current workflow position.

Case	Status	Exit code	Mutation
start/continue before aiqt init	failed	3	None
Canonical files malformed or unsupported	failed	3	None
prompt kind unsupported	failed	3	None
prompt requested in wrong workflow position	blocked	2	None
prompt --out path outside .aiqt/inputs/	failed	3	None
import type unsupported	failed	3	None
import missing --from-file	needs_input	10	None
import file invalid for delegated type	failed	3	None
delegated update/plan/checkpoint blocks	same as delegated command	same as delegated command	same as delegated command
successful start/continue guidance	passed or warning	0	None
successful prompt stdout	passed	0	None
successful prompt --out	passed	0	.aiqt/inputs/ prompt file only
successful import	same as delegated command	same as delegated command	same as delegated command

prompt --out target already exists	blocked	2	None
------------------------------------	---------	---	------

18. Repository Implementation Scope

```

src/
cli/
  commands/
    start.command.ts
    continue.command.ts
    prompt.command.ts
    import.command.ts
  services/
    guidance-service.ts
    prompt-service.ts
    import-service.ts
  templates/
    prompts/
      update.prompt.template.ts
      plan.prompt.template.ts
      checkpoint.prompt.template.ts
  workflow/
    guidance-rules.ts
  schema/
    prompt-result.schema.ts
    guidance-result.schema.ts
tests/
  unit/
  integration/
  fixtures/

```

19. Runtime and Package Impact

- No new runtime dependency is expected for M7.
- Do not add an AI SDK, HTTP client, browser automation library, or agent integration package.
- Use existing filesystem, schema, output, and command registration patterns.
- If a templating helper is useful, prefer simple TypeScript functions over adding a template engine dependency.
- AIQT_SCHEMA_VERSION should remain unchanged because no canonical file schema changes are required.

20. Testing Scope

- Unit tests for guidance state classification.
- Unit tests for prompt rendering and required sections.
- Unit tests for output-path validation under .aiqt/inputs/.
- Integration tests for aiqt start and aiqt continue across initialized, context-ready, planned, in-progress, needs-review, and all-done states.
- Integration tests for prompt update/plan/checkpoint in valid and invalid workflow states.
- Integration tests for prompt --out writing only under .aiqt/inputs/.
- Integration tests for import update/plan/checkpoint delegating to existing behavior.
- Regression tests proving M1-M6 commands still pass unchanged from the M6 baseline: 321/321 tests passing on main before M7.
- Tests proving no markdown source-of-truth files are created.
- Tests proving docs/ remains ignored and uncommitted where repository fixtures model it.

21. Implementation Tasks

Task	Description
M7-T1	Add command registration for start, continue, prompt, and import.
M7-T2	Implement guidance-service and deterministic workflow-stage classification.
M7-T3	Implement prompt-service with update, plan, and checkpoint renderers.
M7-T4	Implement .aiqt/inputs/ path validation and atomic prompt writes.
M7-T5	Implement import-service delegation to existing update, plan, and checkpoint services.
M7-T6	Add runtime result data schemas and JSON output tests.
M7-T7	Add integration tests for complete guided flow from init through prompt/import/next/checkpoint.
M7-T8	Run full validation and preserve M1-M6 behavior.

22. Definition of Done

- aiqt start and aiqt continue return deterministic next-step guidance without mutating files.
- aiqt prompt update, plan, and checkpoint generate useful copy-paste prompts for Claude Code/Codex.
- Prompt commands support --json and optional --out under .aiqt/inputs/.
- Prompt commands reject output paths outside .aiqt/inputs/.
- aiqt import update/plan/checkpoint delegates to existing engine behavior and does not duplicate business logic.
- No API integration or autonomous agent execution is introduced.
- No new canonical files are introduced.
- No root-level markdown files or docs/ files are generated.
- Full validation passes: pnpm typecheck, pnpm lint, pnpm test, pnpm build, pnpm validate.
- M1-M6 tests still pass.

23. Agent Execution Packet

AGENT EXECUTION PACKET - M7

Role

You are implementing AIQT Milestone 7: Guided Workflow and Agent-Assisted Inputs.

Starting Point

M6 is complete and merged to main.

Start from main at commit: bd833ae343f3fbd16ac5747b5605a54ee2861a42

Required existing tag: m6-review-export-engine

M6 validation baseline: 321/321 tests passed before and after merge.

Create branch: feat/m7-guided-workflow

Current Work Unit

Implement the guided workflow UX layer so users can start from an empty folder and use Claude Code/Codex to produce structured AIQT inputs without manually authoring JSON.

Scope

- aiqt start
- aiqt continue
- aiqt prompt update/plan/checkpoint
- aiqt import update/plan/checkpoint --from-file

- .aiqt/inputs/ optional prompt output workspace
- deterministic guidance and prompt rendering
- delegation to existing update/plan/checkpoint services
- tests covering the M7 matrix

Out of Scope

- AI API calls
- autonomous planning
- source-code modification
- hosted backend
- new canonical state files
- markdown source-of-truth files
- changes to M1-M6 behavior

Acceptance Criteria

- User can run aiqt init then aiqt start and receive actionable next-step guidance.
- User can generate prompts for update, plan, and checkpoint inputs.
- User can import agent-generated JSON through aiqt import wrappers.
- start/continue/prompt are read-only except prompt --out writing under .aiqt/inputs/.
- All existing validation passes.

Validation Commands

```

`bash
pnpm typecheck
pnpm lint
pnpm test
pnpm build
pnpm validate
`

```

Required Agent Output

Return:

1. Branch name
2. Commit hash
3. Summary of implementation
4. Files created
5. Files modified
6. Tests added or changed
7. Validation commands run and results
8. Confirmation that M1-M6 tests still pass
9. Confirmation that no out-of-scope functionality was implemented
10. Any unresolved issues
11. Suggested next step

Appendix A - Minimum Test Matrix

Area	Scenario	Expected result
start	.aiqt missing	failed, exit 3, recommends aiqt init, no mutation
start	initialized project with incomplete context	exit 0, recommends aiqt update or aiqt prompt update
start	planning-ready project with empty graph	exit 0, recommends aiqt prompt plan and import plan
start	ready work unit exists	exit 0, recommends aiqt next

start	current work unit in progress	exit 0, recommends aigt prompt checkpoint
start	needs_review exists	exit 0, recommends aigt review
prompt update	initialized project	prints prompt containing update JSON contract
prompt plan	planningContextReady false	blocked, exit 2, recommends aigt update
prompt plan	planningContextReady true and graph empty	prints prompt containing plan JSON contract
prompt checkpoint	no current in_progress work unit	blocked, exit 2, recommends aigt next
prompt checkpoint	current in_progress work unit exists	prints prompt containing checkpoint JSON contract
prompt --out	path under .aiqt/inputs/	writes prompt file and no canonical state mutation
prompt --out	path outside .aiqt/inputs/	failed, exit 3, no mutation
import update	valid update file	same behavior as aigt update --from-file
import plan	valid plan file	same behavior as aigt plan --from-file
import checkpoint	valid checkpoint file	same behavior as aigt checkpoint --from-file
import	missing --from-file	needs_input, exit 10
regression	M1-M6 validation suite from main baseline	321/321 existing tests pass before M7 additions; all tests pass after M7 additions
artifact discipline	commands do not create AIQT.md/AGENTS.md/CLAUDE.md/docs/.milestones/.tasks	pass
start/continue	currentWorkUnitId non-null and needs_review work also exists	uses priority order; recommends aigt checkpoint, exit 0, no mutation
guidance	recommendation computed for workflow stage	reuses state.nextRecommendedCommand / existing workflow helpers / M6 precedence where applicable; no divergent workflow engine
guidance data	canProceedWithoutAgent values	true for user-direct actions; false when agent-assisted structured input is normally required
prompt --out	target file already exists	blocked, exit 2, no overwrite, recommends same prompt command with different --out path

Appendix B - Example Generated Prompts

B.1 aigt prompt plan - abbreviated

You are helping create an AIQT structured plan input.
Return JSON only. Do not wrap the JSON in markdown.

Project objective:
<objective from project.json>

Target users:
<target users>

Relevant context:

- Constraints: <constraints>
- Technology preferences: <technologyPreferences>
- Business rules: <businessRules>
- Requirements: <requirements>

- Decisions: <decisions>

Create a plan JSON with this shape:

```
{
  "milestones": [
    { "clientKey": "...", "title": "...", "objective": "..." }
  ],
  "workUnits": [
    {
      "clientKey": "...",
      "milestoneClientKey": "...",
      "title": "...",
      "objective": "...",
      "scope": ["..."],
      "outOfScope": ["..."],
      "acceptanceCriteria": ["..."],
      "agentContextRefs": [],
      "suggestedFiles": ["..."],
      "validationCommands": ["..."]
    }
  ],
  "dependencies": []
}
```

Save the returned JSON to .aiqt/inputs/plan.json.

Then run: `aiqt import plan --from-file .aiqt/inputs/plan.json`

Appendix C - Example JSON Outputs

C.1 aiqt start --json - needs plan

```
{
  "status": "passed",
  "action": "start",
  "summary": "Project context is ready. Generate a structured plan next.",
  "nextRecommendedCommand": "aiqt prompt plan",
  "data": {
    "stage": "needs_plan",
    "guidance": "Generate a plan prompt, ask Claude Code or Codex to return plan JSON, then import it.",
    "recommendedCommand": "aiqt prompt plan",
    "alternativeCommands": ["aiqt plan --from-file .aiqt/inputs/plan.json"],
    "promptCommand": "aiqt prompt plan --out .aiqt/inputs/plan.prompt.md",
    "expectedInputPath": ".aiqt/inputs/plan.json",
    "followUpCommand": "aiqt import plan --from-file .aiqt/inputs/plan.json",
    "canProceedWithoutAgent": false
  },
  "exitCode": 0
}
```